

Windows Forms

CHAPTER 19

*Pro C# with .NET 3.0, Andrew Troelsen, Special Edition,
Dream tech Press, India, 2007.*

Objectives

- the process of building traditional form-based desktop applications.
- build a highly stylized main window using the Form and Application classes.
- how to capture and respond to user input (i.e., handle mouse and keyboard events) within the context of a GUI desktop environment.
- to construct menu systems, toolbars, status bars, and multiple-document interface (MDI) applications, both by hand and using the designers incorporated into Visual Studio 2008.

Overview of the System.Windows.Forms Namespace

- Like any namespace, System.Windows.Forms is composed of various classes, structures, delegates, interfaces, and enumerations.
- The process of building a Windows Forms application involves nothing more than learning how to manipulate a new set of types using the C# syntax you already know.

- From a high level, the hundreds of types within the System.Windows.Forms namespace can be grouped into the following broad categories:

- **Core infrastructure**

These are types that represent the core operations of a .NET Forms program

(Form, Application, etc.) and various types to facilitate interoperability with legacy ActiveX controls.

- **Controls**

These are types used to create rich UIs (Button, MenuStrip, ProgressBar,

DataGridView, etc.), all of which derive from the Control base class.

Controls are configurable at design time and are visible (by default) at runtime.

- **Components**

These are types that do not derive from the Control base class but still provide visual features to a .NET Forms program (ToolTip, ErrorProvider, etc.).

Many components (such as the Timer) are not visible at runtime, but can be configured visually at design time.

- **Common dialog boxes**

Windows Forms provides a number of canned dialog boxes for common operations (OpenFileDialog, PrintDialog, etc.).

As you would hope, you can certainly build your own custom dialog boxes if the standard dialog boxes do not suit your needs.

Core .NET 2.0 System.Windows.Forms types

Classes	Meaning in Life
Application	This class encapsulates the runtime operation of a Windows Forms application.
Button, CheckBox, ComboBox, DateTimePicker, ListBox, LinkLabel, MaskedTextBox, MonthCalendar, PictureBox, TreeView	These classes (in addition to many others) correspond to various GUI widgets. You'll examine many of these items in detail in Chapter 21.
FlowLayoutPanel, TableLayoutPanel	.NET 2.0 now supplies various "layout managers" that automatically arrange a Form's controls during resizing.
Form	This type represents a main window, dialog box, or MDI child window of a Windows Forms application.
ColorDialog, OpenFileDialog, SaveFileDialog, FontDialog, PrintPreviewDialog, FolderBrowserDialog	These are various standard dialog boxes for common GUI operations.
Menu, MainMenu, MenuItem, ContextMenu, MenuStrip, ContextMenuStrip,	These types are used to build topmost and context-sensitive menu systems. The new (.NET 2.0) MenuStrip and ContextMenuStrip controls allow you to build menus that may contain traditional drop-down menu items as well as other controls (text boxes, combo boxes, and so forth).
StatusBar, Splitter, ToolBar, ScrollBar, StatusStrip, ToolStrip	These types are used to adorn a Form with common child controls.

Working with the Windows Forms Types

- When you build a Windows Forms application, you may choose to write all the relevant code by hand (using Notepad or TextPad, perhaps) and feed the resulting *.cs files into the C# compiler using the /target:winexe flag.
- Taking time to build some Windows Forms applications by hand not only is a great learning experience, but also helps you understand the code generated by the various graphics designers found within various .NET IDEs.

Building a Main Window by Hand

- Create a new folder on your hard drive (e.g., C:\MyFirstWindow) and create a new file within this directory named MainWindow.cs using your editor of choice.
- In the world of Windows Forms, the Form class is used to represent any window in your application.
- This includes a topmost main window in a single-document interface (SDI) application, modeless and modal dialog boxes, and the parent and child windows of a multiple-document interface (MDI) application.

- When you are interested in creating and displaying the main window in your program, you have two mandatory steps:
 1. Derive a new class from `System.Windows.Forms.Form`.
 2. Configure your application's `Main()` method to invoke `Application.Run()`, passing an instance of your Form-derived type as an argument.
- Given this, update your `MainWindow.cs` file with the following class definition:

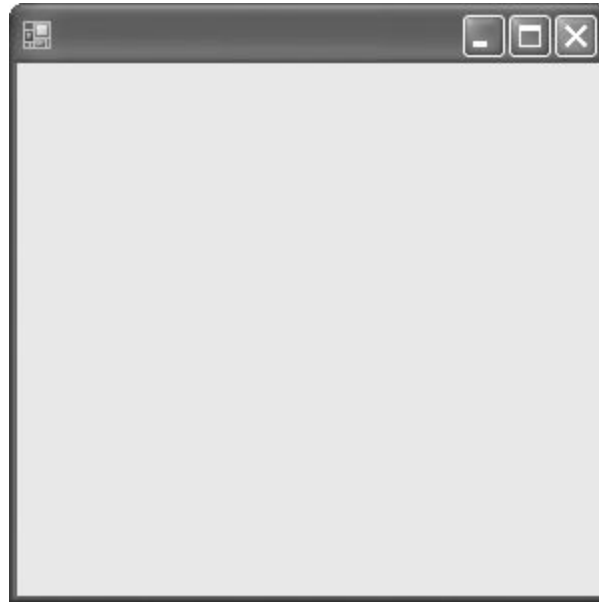
```
using System;
using System.Windows.Forms;
namespace MyWindowsApp
{
    public class MainWindow : Form
    {
        // Run this application and identify the main window.
        static void Main(string[] args)
        {
            Application.Run(new MainWindow());
        }
    }
}
```

- In addition to the always present mscorlib.dll, a Windows Forms application needs to reference
- the System.dll and System.Windows.Forms.dll assemblies.
- As you may recall, the default C# response file (csc.rsp) instructs csc.exe to automatically include these assemblies during the compilation process.
- Also recall that the /target:winexe option of csc.exe instructs the compiler to generate a Windows executable.

- To compile your C# code file, open a Visual Studio 2008 command prompt and issue the following command:

```
csc /target:winexe *.cs
```

A simple main window of Windows Forms



Honoring the Separation of Concerns

- Currently, the MainWindow class defines the Main() method directly within its scope.
- you may create a second static class, say Program) that is responsible for the task of launching the main window, leaving the Form-derived class responsible for representing the window itself:

-

```
namespace MyWindowsApp
{
    // The main window.
    public class MainWindow : Form { }
    // The application object.
    public static class Program
    {
        static void Main(string[] args)
        {
            // Don't forget to 'use' System.Windows.Forms!
            Application.Run(new MainWindow());
        }
    }
}
```

- By doing so, you are abiding by an OO principal termed *the separation of concerns*.
- *Simply put,*
- this rule of OO design states that a class should be in charge of doing the least amount of work possible.
- Given that you have refactored the initial class into two unique classes, you have decoupled the Form from the class that creates it.
- The end result is a more portable window, as it can be dropped into any project without carrying the extra baggage of a project-specific Main() method.

The Role of the Application Class

- The Application class defines numerous static members that allow to control various low-level behaviors of a Windows Forms application.
 - For example, the Application class defines a set of events that allow to respond to events such as application shutdown and idle-time processing.
- In addition to the Run() method, here are some other methods to be aware of:
- DoEvents(): Provides the ability for an application to process messages currently in the message queue during a lengthy operation.
- Exit(): Terminates the Windows application and unloads the hosting AppDomain.
- EnableVisualStyles(): Configures your application to support Windows XP visual styles.

Core Properties of the Application Type

Property	Meaning in Life
CompanyName	Retrieves the value of the assembly-level [AssemblyCompany] attribute
ExecutablePath	Gets the path for the executable file
ProductName	Retrieves the value of the assembly-level [AssemblyProduct] attribute
ProductVersion	Retrieves the value of the assembly-level [AssemblyVersion] attribute
StartupPath	Retrieves the path for the executable file that started the application

- The Application class defines various static events, some of which are as follows:
- ApplicationExit: Occurs when the application is just about to shut down
- Idle: Occurs when the application's message loop has finished processing the current batch of messages and is about to enter an idle state (as there are no messages to process at the current time)
- ThreadExit: Occurs when a thread in the application is about to terminate

Illustration of the functionality of the Application class

- Let's enhance your current MainWindow to perform the following:
- Reflect over select assembly-level attributes.
- Handle the static ApplicationExit event.
- The first task is to make use of select properties in the Application class to reflect over some assembly-level attributes.
- To begin, add the following attributes to your MainWindow.cs file (note you are now using the System.Reflection namespace):
 - using System;
 - using System.Windows.Forms;
 - using System.Reflection;

- **// Some attributes regarding this assembly.**
[assembly:AssemblyCompany("Intertech Training")]
[assembly:AssemblyProduct("A Better Window")]
[assembly:AssemblyVersion("1.1.0.0")]
namespace MyWindowsApp
{
...
}

- Rather than manually reflecting over the [AssemblyCompany] and [AssemblyProduct] attributes, the Application class will do so automatically using various static properties.
- To illustrate, implement the default constructor of MainForm as so:

```
public class MainWindow : Form
{
    public MainWindow()
    {
        MessageBox.Show(Application.ProductName,
            string.Format("This app brought to you by {0}",
                Application.CompanyName));
    }
}
```

When this application is executed, a message box is displayed with various bits of information



- Now, equip this Form to respond to the ApplicationExit event.
- To intercept the static ApplicationExit event, simply register an event handler using the += operator:

```
public class MainForm : Form
{
    public MainForm()
    {
        ...
        // Intercept the ApplicationExit event.
        Application.ApplicationExit += new EventHandler(MainWindow_OnExit);
    }
    private void MainWindow_OnExit(object sender, EventArgs evArgs)
    {
        MessageBox.Show(string.Format("Form version {0} has terminated.",
        Application.ProductVersion));
    }
}
```

The **System.EventHandler Delegate**

- The **ApplicationExit** event works in conjunction with the **System.EventHandler** delegate.
- This delegate must point to methods that conform to the following signature:

delegate void **EventHandler(object sender, EventArgs e);**

- `System.EventHandler` is the most primitive delegate used to handle events within Windows Forms, but many variations do exist for other events.
- As far as `EventHandler` is concerned, the first parameter of the assigned method is of type `System.Object`, which represents the object sending the event.
- The second `EventArgs` parameter (or a descendent thereof) contains any relevant information regarding the current event.

- EventArgs is the base class to numerous derived types that contain information for a family of related events.
- For example, mouse events work with the MouseEventArgs parameter, which contains details such as the (x, y) position of the cursor.
- Many keyboard events work with the KeyEventArgs type, which contains details regarding the current keypress, and so forth.

The Anatomy of a Form

- The Form class inherits a great deal of functionality from its parent classes.

Figure shows the inheritance chain (including the set of implemented interfaces) of a Form-derived type using the Visual Studio 2008 Object Browser.

The screenshot displays the Visual Studio 2008 Object Browser window. The 'Browse:' dropdown is set to 'My Solution'. The search bar contains '<Search>'. The left pane shows a tree view of the inheritance chain for the **Form** class, which is highlighted. The tree structure is as follows:

- Form**
 - Base Types
 - ContainerControl
 - IContainerControl
 - ScrollableControl
 - Control
 - Component
 - IComponent
 - IDisposable
 - MarshalByRefObject
 - Object
 - IBindableComponent
 - IComponent
 - IDisposable
 - IDropTarget
 - ISynchronizeInvoke
 - IWin32Window
 - IComponent
 - IDisposable

The right pane lists the methods implemented by the **Form** class:

 - Activate()
 - AddOwnedForm(System.Windows.Forms.Form)
 - Close()
 - Form()
 - LayoutMdi(System.Windows.Forms.MdiLayout)
 - ProcessMnemonic(char)
 - RemoveOwnedForm(System.Windows.Forms.Form)
 - SetDesktopBounds(int, int, int, int)
 - SetDesktopLocation(int, int)
 - Show(System.Windows.Forms.IWin32Window)
 - ShowDialog(System.Windows.Forms.IWin32Window)
 - ShowDialog()

Below the method list, the class declaration is shown:

```
public class Form : System.Windows.Forms.ContainerControl  
    Member of System.Windows.Forms
```

Summary:
Represents a window or dialog box that makes up an application's user interface.

- Although the complete derivation of a Form type involves numerous base classes and interfaces, it is *not required to learn the role of each and every member from each and every* parent class or implemented interface to be a proficient Windows Forms developer.
- In fact, the majority of the members (properties and events in particular) are easily set using the Visual Studio 2008 IDE Properties window.
- Table (next slide) outlines the basic role of each base class.

Parent Class

Meaning in Life

- `System.Object` Like any class in .NET, a Form “is-a” object.
- `System.MarshalByRefObject` Recall during our examination of .NET remoting that types deriving from this class are accessed remotely via a *reference* (*not* a copy) of the remote type.
- `System.ComponentModel.Component` This class provides a default implementation of the `IComponent` interface. In the .NET universe, a component is a type that supports design-time editing, but is not necessarily visible at runtime.
- `System.Windows.Forms.Control` This class defines common UI members for all Windows Forms UI controls, including the `Form` type itself.
- `System.Windows.Forms.ScrollableControl` This class defines support for auto-scrolling behaviors.
- `System.Windows.Forms.ContainerControl` This class provides focus-management functionality for controls that can function as a container for other controls.
- `System.Windows.Forms.Form` This class represents any custom Form, MDI child, or dialog box.

The Functionality of the Control Class

- The `System.Windows.Forms.Control` class establishes the common behaviors required by any GUI type.
- The core members of `Control` allow to configure the size and position of a control, capture keyboard and mouse input, get or set the focus/visibility of a member, and so forth.
- Table (next slide) defines some (but not all) properties of interest, grouped by related functionality

Core Properties of the Control Type

- BackColor, ForeColor, BackgroundImage, Font, Cursor
 - These properties define the core UI of the control (colors, font for text, mouse cursor to display when the mouse is over the widget, etc.).

- Anchor, Dock, AutoSize
 - These properties control how the control should be positioned within the container.

- Top, Left, Bottom, Right, Bounds, the ClientRectangle, Height, Width. Enabled, Focused, Visible
 - These properties specify the current dimensions of control.
 - These properties each return a Boolean that specifies the state of the current control.

- **ModifierKeys**

- This static property checks the current state of the modifier keys (Shift, Ctrl, and Alt) and returns the state in a Keys type.

- **MouseButtons**

- This static property checks the current state of the mouse buttons (left, right, and middle mouse buttons) and returns this state in a MouseButtons type.

- TabIndex, TabStop
 - These properties are used to configure the tab order of the control.

- Opacity
 - This property determines the opacity of the control, in fractions (0.0 is completely transparent; 1.0 is completely opaque).

- Text

- This property indicates the string data associated with this control.

- Controls

- This property allows you to access a strongly typed collection (ControlsCollection) that contains any child controls within the current control.

- The Control class also defines a number of events that allow you to intercept mouse, keyboard, painting, and drag-and-drop activities (among other things).
- Table (next slide) lists some (but not all) events of interest, grouped by related functionality.

Events and their Meanings in Life

- Click, DoubleClick, MouseEnter, MouseLeave, MouseDown, MouseUp, MouseMove, MouseHover, MouseWheel
 - Various events that allow you to interact with the mouse

- KeyPress, KeyUp, KeyDown
 - Various events that allow you to interact with the keyboard
- DragDrop, DragEnter, DragLeave, DragOver
 - Various events used to monitor drag-and-drop activity
- Paint
 - This event allows you to interact with GDI (Graphical device interface)

- Finally, the Control base class also defines a number of methods that allow to interact with any Control-derived type.
- The methods of the Control type have an On prefix followed by the name of a specific event (OnMouseMove, OnKeyUp, OnPaint, etc.).
- Each of these On-prefixed virtual methods is the default event handler for its respective event.

```
public class MainWindow : Form
{
    public MainWindow()
    {
        MouseDown += new
            MouseEventHandler(MainWindow_MouseDown);
    }

    void MainWindow_MouseDown(object sender,
        MouseEventArgs e)
    {
        // Add code for MouseDown event.
    }
}
```

- Beyond these OnXXX() methods, here are a few other methods to be aware of:
- Hide(): Hides the control and sets the Visible property to false
- Show(): Shows the control and sets the Visible property to true
- Invalidate(): Forces the control to redraw itself by sending a Paint event

- To illustrate the usefulness of some members from the Control class, let's build a new Form that is capable of handling the following events:
- Respond to the MouseMove and MouseDown events.
- Capture and process keyboard input via the KeyUp event.

- To begin, create a new class derived from Form.
- In the default constructor, make use of various inherited properties to establish the initial look and feel.


```
using System;
using System.Windows.Forms;
using System.Drawing;
namespace MyWindowsApp
{
    public class MainWindow : Form
    {
        public MainWindow()
        {
            // Use inherited properties to set basic UI.
            Text = "My Fantastic Form";
            Height = 300;
            Width = 500;
            BackColor = Color.LemonChiffon;
            Cursor = Cursors.Hand;
        }
    }
    public static class Program
    {
        static void Main(string[] args)
        {
            Application.Run(new MainWindow());
        }
    }
}
```

Responding to the MouseMove Event

- Handling the MouseMove event.
 - The goal is to display the current (x, y) location within the Form's caption area.
- All mouse-centric events (MouseMove, MouseUp, etc.) work in conjunction with the MouseEventArgs delegate, which can call any method matching the following signature:
 - `void MyMouseHandler(object sender, MouseEventArgs e);`

- The incoming MouseEventArgs structure extends the general EventArgs base class by adding a number of members particular to the processing of mouse activity (see Table (next slide)).

Properties of the MouseEventArgs Type

- Button
 - Gets which mouse button was pressed, as defined by the MouseButton enumeration
- Clicks
 - Gets the number of times the mouse button was pressed and released
- Delta
 - Gets a signed count of the number of detents the mouse wheel has rotated
- X
 - Gets the x-coordinate of a mouse click
- Y
 - Gets the y-coordinate of a mouse click

Here, then, is the updated MainForm class that handles the MouseMove event as intended:

```
public class MainForm : Form
{
    public MainForm()
    {
        ...
        // Handle the MouseMove event
        MouseMove += new MouseEventHandler(MainForm_MouseMove);
    }
    // MouseMove event handler.
    public void MainForm_MouseMove(object sender, MouseEventArgs e)
    {
        Text = string.Format("Current Pos: ({0}, {1})", e.X, e.Y);
    }
}
```

- If you now run your program and move the mouse over your Form, you will find the current (x, y) value display on the caption area



Current Pos: (106, 63)



Determining Which Mouse Button Was Clicked

- One thing to be aware of is that the MouseUp (or MouseDown) event is sent whenever any mouse button is clicked.
- To determine exactly which button was clicked (such as left, right, or middle), examine the Button property of the MouseEventArgs class.
- The value of the Button property is constrained by the related MouseButton enumeration.


```
public MainWindow()  
{  
...  
// Handle the MouseUp event.  
MouseUp += new MouseEventHandler(MainForm_MouseUp);  
}
```

The following MouseUp event handler displays which mouse button was clicked inside
a message box:

```
public void MainForm_MouseUp (object sender, MouseEventArgs e)  
{  
// Which mouse button was clicked?  
if(e.Button == MouseButton.Left)  
    MessageBox.Show("Left click!");  
if(e.Button == MouseButton.Right)  
    MessageBox.Show("Right click!");  
if (e.Button == MouseButton.Middle)  
    MessageBox.Show("Middle click!");  
}
```

Responding to Keyboard Events

- Processing keyboard input is almost identical to responding to mouse activity.
- The KeyUp and KeyDown events work in conjunction with the KeyEventHandler delegate, which can point to any method taking an object as the first parameter and KeyEventArgs as the second:
 - `void MyKeyboardHandler(object sender, KeyEventArgs e);`
- KeyEventArgs has the members of interest shown in Table (next slide)

- Alt
 - Gets a value indicating whether the Alt key was pressed
- Control
 - Gets a value indicating whether the Ctrl key was pressed
- Handled
 - Gets or sets a value indicating whether the event was fully handled in your handler
- KeyCode
 - Gets the keyboard code for a KeyDown or KeyUp event
- Modifiers
 - Indicates which modifier keys (Ctrl, Shift, and/or Alt) were pressed
- Shift
 - Gets a value indicating whether the Shift key was pressed

- Update the MainForm to handle the KeyUp event.
- Then, display the name of the key that was pressed inside a message box using the KeyCode property.

```
public class MainForm : Form
```

```
{
```

```
public MainForm()
```

```
{
```

```
...
```

```
// Listen for the KeyUp Event.
```

```
KeyUp += new KeyEventHandler(MainForm_KeyUp);
```

```
}
```

```
private void MainForm_KeyUp (object sender, KeyEventArgs e)
```

```
{
```

```
MessageBox.Show(e.KeyCode.ToString(), "Key Pressed!");
```

```
}
```

```
}
```

- Now compile and run the program.
- You should be able to determine not only which mouse button was clicked, but also which keyboard key was pressed.

The Functionality of the Form Class

- The Form class is typically (but not necessarily) the direct base class for the custom Form types.
- In addition to the large set of members inherited from the Control, ScrollableControl, and ContainerControl classes, the Form type adds additional functionality in particular to main windows, MDI child windows, and dialog boxes.

Properties of the Form Type

- `AcceptButton`
 - Gets or sets the button on the Form that is clicked when the user presses the Enter key.
- `ActiveMDIChild`
- `IsMDIChild`
- `IsMDIContainer`
 - Used within the context of an MDI application.

- CancelButton

- Gets or sets the button control that will be clicked when the user presses the Esc key.

- **ControlBox**
 - Gets or sets a value indicating whether the Form has a control box.
- **FormBorderStyle**
 - Gets or sets the border style of the Form. Used in conjunction with the FormBorderStyle enumeration.
- **Menu**
 - Gets or sets the menu to dock on the Form.

- MaximizeBox
- MinimizeBox
 - Used to determine if this Form will enable the maximize and minimize boxes.

- ShowInTaskbar
 - Determines if this Form will be seen on the Windows taskbar.
- StartPosition
 - Gets or sets the starting position of the Form at runtime, as specified by the FormStartPosition enumeration.

- WindowState
 - Configures how the Form is to be displayed on startup. Used in conjunction with the FormWindowState enumeration.

Key Methods of the Form Type

- `Activate()`
 - Activates a given Form and gives it focus.
- `Close()`
 - Closes a Form.
- `CenterToScreen()`
 - Places the Form in the dead-center of the screen.
- `LayoutMDI()`
 - Arranges each child Form (as specified by the `LayoutMDI` enumeration) within the parent Form.
- `ShowDialog()`
 - Displays a Form as a modal dialog box. More on dialog box programming

Select Events of the Form Type

- Activated
 - Occurs whenever the Form is *activated*, meaning the Form has been given the current focus on the desktop
- Closed, Closing
 - Used to determine when the Form is about to close or has closed
- Deactivate
 - Occurs whenever the Form is *deactivated*, meaning the Form has lost current focus on the desktop
- Load
 - Occurs after the Form has been allocated into memory, but is not yet visible on the screen
- MDIChildActive
 - Sent when a child window is activated

Building Windows Applications with Visual Studio 2008

- Visual Studio 2008 has a specific project type dedicated to the creation of Windows Forms applications.
- When you select the Windows Application project type, you not only receive an application object with a proper `Main()` method, but also are provided with an initial Form-derived type.
- Better yet, the IDE provides a number of graphical designers that make the process of building a UI child's play.

New Project



Project types:

Templates:



- [-] Visual C #
 - ... Windows
 - [+] Smart Device
 - ... Database
 - ... Starter Kits
 - ... Test
 - ... Distributed System Solutions
- [+] Other Languages
- [+] Other Project Types
- [+] Test Projects

Visual Studio installed templates



Windows Application

Class Library

Windows Contr...

Console Application

Device Application

My Templates



Search Online Templates...

A project for creating an application with a Windows user interface

Name:

MyVisualStudioWinApp

Location:

C:\Documents and Settings\Andrew Troelsen\My Documents\Visual Studio 2005\Projects

Browse...

Solution Name:

MyVisualStudioWinApp

☐ Create directory for solution

☐ Add to Source Control

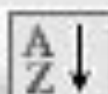
- Once the project has loaded, you will notice the Forms designer, which allows to build a UI by dragging controls/components from the Toolbox and configuring their properties and events using the Properties window.



Properties



MainWindow System.Windows.Forms.Form



☒ (ApplicationSettings)



☒ (DataBindings)



(Name) **MainWindow**

AcceptButton (none)

AccessibleDescription

AccessibleName

AccessibleRole Default

AllowDrop False

AutoScaleMode **Font**

AutoScroll False

☒ AutoScrollMargin 0, 0

☒ AutoScrollMinSize 0, 0



- the Toolbox groups UI controls by various categories. While most are selfexplanatory
 - (e.g., Printing contains printing controls, Menus & Toolbars contains recommended menu/toolbar controls, etc.),
- A few categories deserve special mention:
- Common Controls: Members in this category are considered the “recommended set” of common UI controls.
- All Windows Forms: Here you will find the full set of Windows Forms controls, including various .NET 1.x controls that are considered depreciated.

Enabling the Deprecated Controls

- These (deprecated) UI elements are usable under .NET 2.0.

Choose Toolbox Items



.NET Framework Components COM Components

Name	Namespace	Directory
☒ Timer	System.Windows.Forms	C:\Program Files\Microsoft Visual Studi
☒ Timer	System.Windows.Forms	C:\Program Files\Microsoft Visual Studi
☐ TimerArray	Microsoft.VisualBasic.Compatibi...	Global Assembly Cache
☒ ToolBar	System.Windows.Forms	Global Assembly Cache
☒ ToolBar	System.Windows.Forms	C:\Program Files\Microsoft Visual Studi
☒ ToolBar	System.Windows.Forms	C:\Program Files\Microsoft Visual Studi
☐ ToolBarArray	Microsoft.VisualBasic.Compatibi...	Global Assembly Cache
☒ ToolStrip	System.Windows.Forms	Global Assembly Cache
☐ ToolStripArray	Microsoft.VisualBasic.Compatibi...	Global Assembly Cache

Filter:

Clear

ToolBar



Language: Invariant Language (Invariant Country)

Version: 2.0.0.0

Browse...

OK

Cancel

Reset

Dissecting a Visual Studio 2008 Windows Forms Project

- Each Form in a Visual Studio 2008 Windows Application project is composed of two related C# files, which can be verified using Solution Explorer

Solution Explorer - Solution 'TestWin'...



 Solution 'TestWin' (1 project)

 **TestWin**

 Properties

 References

 **Form1.cs**

 Form1.Designer.cs

 Program.cs



Solution Explorer



Team Explorer

- Right-click the Form1.cs icon and select View Code.
- Here you will see a partial class that contains all of the Form's event handlers, constructors, overrides, and any member you author yourself (note that I renamed this initial class from Form1 to MainWindow using the Rename refactoring):

```
namespace MyVisualStudioWinApp
{
    public partial class MainWindow : Form
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- The default constructor of your Form makes a call to a method named `InitializeComponent()`, which is defined within the related `*.Designer.cs` file.
- This method is maintained by Visual Studio 2008, and it contains all of the code representing your design-time modifications.
- To illustrate, switch back to the Forms designer and locate the Text property in the Properties window.
- Change this value to something like **My Test Window**. **Now open your Form1.Designer.cs** file and notice that `InitializeComponent()` has been updated accordingly:

```
private void InitializeComponent()  
{  
...  
this.Text = "My Test Window";  
}
```

Handling Events at Design Time

- Notice that the Properties window has a button depicting a lightning bolt.
- Although Form-level events are always free to handle by authoring the necessary logic by hand , this event button allows to visually handle an event for a given control.
- Simply select the control you wish to interact with from the drop-down list box (mounted at the top of the Properties
- window), locate the event you are interested in handling, and type in the name to be used as an event handler (or simply double-click the event to generate a default name of the form ControlName_EventName).

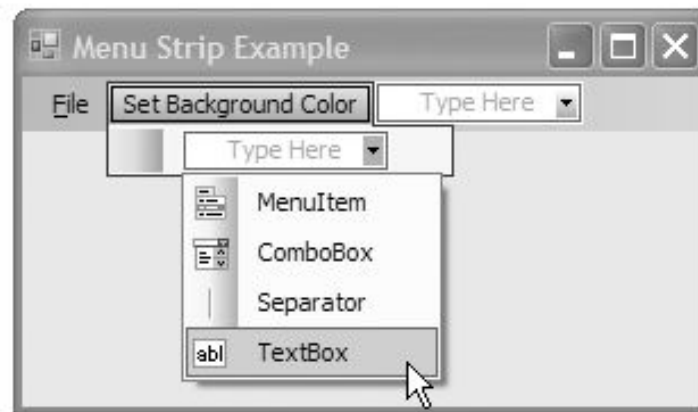
Working with MenuStrips and ContextMenuStrips

- The recommended control for building a menu system is MenuStrip.
- This control allows you to create “normal” menu items such as File ->**Exit**,
- **It may be configured to contain** any number of relevant controls within the menu area.
- Some common UI elements that may be contained within a MenuStrip:
 - ToolStripMenuItem: A traditional menu item
 - ToolStripComboBox: An embedded ComboBox
 - ToolStripSeparator: A simple line that separates content
 - ToolStripTextBox: An embedded TextBox

Adding a TextBox to the MenuStrip

- Now, let's create a new topmost menu item named Change Background Color.
- The subitem in this case will not be a menu item, but a ToolStripTextBox

Form1.Designer.cs* Form1.cs* **Form1.cs [Design]*** ▾ ✕



menuStrip1

- Once the new control is added, rename this control to `toolStripTextBoxColor` using the Properties window.
- The goal here is to allow the user to enter the name of a color (red, green, pink, etc.) that will be used to set the `BackColor` property of the Form.
- First, handle the `LostFocus` event on the new `ToolStripTextBox` member variable within the Form's constructor (this event fires when the `TextBox` within the `ToolStrip` is no longer the active UI element):

```
public MainWindow()  
{  
...  
toolStripTextBoxColor.LostFocus  
+= new EventHandler(toolStripTextBoxColor_LostFocus);  
}
```

- Within the generated event handler, you will extract the string data entered within the ToolStripTextBox (via the Text property) and make use of the `System.Drawing.Color.FromName()` method.
- This static method will return a Color type based on a known string value.
- To account for the possibility that the user enters an unknown color (or types bogus data), you will make use of some simple try/catch logic:

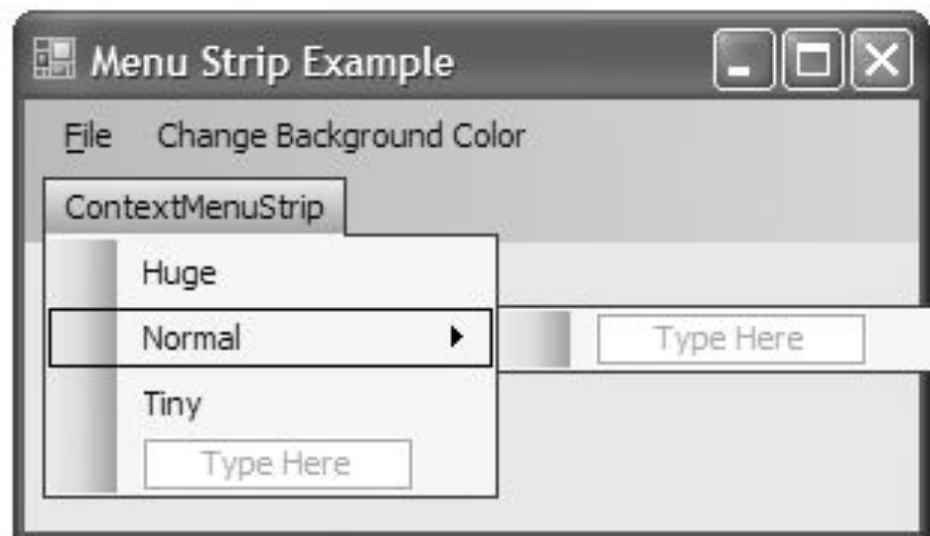
```
void toolStripTextBoxColor_LostFocus(object sender,  
    EventArgs e)  
{  
    try  
    {  
        BackColor = Color.FromName(toolStripTextBoxColor.Text);  
    } catch { } // Just do nothing if the user provides bad data.  
}
```

Creating a Context Menu

- Let's now examine the process of building a context-sensitive pop-up (i.e., right-click) menu.
- Under .NET 1.1, the ContextMenu type was the class of choice for building context menus, but under .NET 2.0 the preferred type is ContextMenuStrip.
- Like the MenuStrip type, ContextMenuStrip maintains a ToolStripItemCollection to represent the possible subitems (such as ToolStripMenuItem, ToolStripComboBox, ToolStripSeparator, ToolStripTextBox, etc.).

- Drag a new ContextMenuStrip control from the Toolbox onto the Forms designer and rename the control to fontSizeContextStrip using the Properties window.
- Notice that you are able to populate the subitems graphically in much the same way you would edit the Form's main MenuStrip (a welcome change from the method used in Visual Studio .NET 2003).
- For this example, add three ToolStripMenuItem named Huge, Normal, and Tiny

Form1.cs [Design]* Object Browser



menuStrip1 fontSizeContextStrip

- This context menu will be used to allow the user to select the size to render a message within the Form's client area.
- To facilitate this endeavor, create an enum type named `TextFontSize` within the `MenuStripApp` namespace and declare a new member variable of this type within your `Form` type (set to `TextFontSize.FontSizeNormal`):


```
namespace MainForm
{
// Helper enum for font size.
enum TextFontSize
{
    FontSizeHuge = 30,
    FontSizeNormal = 20,
    FontSizeTiny = 8
}
public class MainForm : Form
{
// Current size of font.
    private TextFontSize currFontSize
    = TextFontSize.FontSizeNormal;
    ...
}
}
```

- The next step is to handle the Form's Paint event using the Properties window.
- The Paint event allows you to render graphical data (including stylized text) onto a Form's client area.
- Here, you are going to draw a textual message using a font of userspecified size.

```
private void MainWindow_Paint(object sender,  
    PaintEventArgs e)  
{  
    Graphics g = e.Graphics;  
    g.DrawString("Right click on me...",  
        new Font("Times New Roman", (float)currFontSize),  
        new SolidBrush(Color.Black), 50, 50);  
}
```

Checking Menu Items

- ToolStripMenuItem defines a number of members that allow you to check, enable, and hide a given item.

Members of the ToolStripMenuItem Type

- | • Member | Meaning in Life |
|---|---|
| • Checked
indicating whether a
ToolStripMenuItem | Gets or sets a value
beside the text of the
check mark appears |
| • CheckOnClick
indicating whether the
should | Gets or sets a value
ToolStripMenuItem
automatically appear
checked/unchecked when clicked |
| • Enabled
whether the ToolStripMenuItem is | Gets or sets a value indicating
enabled |

Working with StatusStrips

- In addition to a menu system, many Forms also maintain a *status bar that is typically mounted at the bottom of the Form*.
- A status bar may be divided into any number of “panes” that hold some textual (or graphical) information such as menu help strings, the current time, or other applicationspecific information.
- Although status bars have been supported since the release of the .NET platform (via the `System.Windows.Forms.StatusBar` type), as of .NET 2.0 the simple `StatusBar` has been ousted by the new `StatusStrip` type.
- Like a status bar, a `StatusStrip` can consist of any number of panes to hold textual/graphical data using a `ToolStripStatusLabel` type.